# AuraAI

## AI Learning Platform
## Technical Documentation Set

SENIOR TECHNICAL WRITER & SOFTWARE ARCHITECT

**Abiram S.**

August 2026

# Table of Contents

# 1. Product Requirements Document (PRD)

## 1.1 Executive Summary

The core vision of **AuraAI** is to democratize education through an intelligent, highly engaging AI Learning Platform. AuraAI aims to bridge the gap between static e-learning and interactive education by incorporating gamification (XP, levels, and streaks), interactive assessments, and real-time feedback loops between students and instructors. By leveraging modern web technologies, the platform provides a seamless, dynamic, and responsive experience for all educational stakeholders.

## 1.2 User Roles & Target Audience

AuraAI caters to three distinct user personas, each with highly specialized access protocols and dashboard environments:

- **Student:** The primary consumer of the platform. Students can browse the catalog, enroll in Free or Paid (₹) courses, consume learning modules, and complete interactive multiple-choice quizzes. Their engagement is incentivized via an XP and leveling system, alongside a star-rating feature to review instructors on a competitive leaderboard.
- **Instructor (Teacher):** Content creators and classroom managers. Instructors operate within a restricted portal allowing them to build courses, author quizzes, and track daily student attendance. They manage their public profile, which displays their aggregated ratings and scheduled class check-ins (e.g., 9:00 AM daily syncs).
- **Admin:** Platform overseers responsible for moderation and user management. Admins possess global privileges to audit the course catalog (add/delete), monitor daily login metrics for both students and instructors, and manually provision new student accounts.

## 1.3 Key Feature Specifications

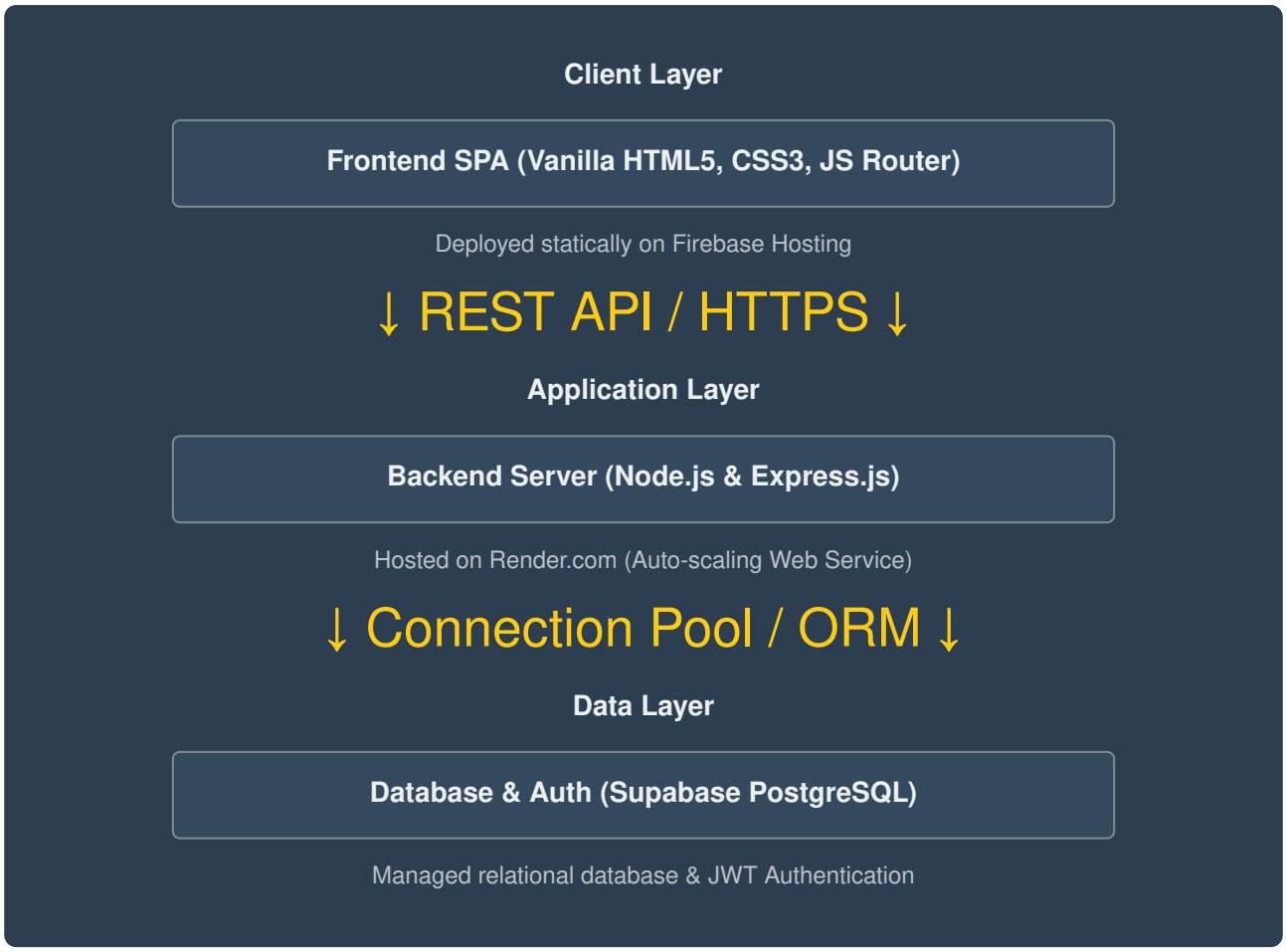
- **Course Catalog:** A rich, filterable grid displaying available courses with details on difficulty, pricing, and category.
- **Interactive Quiz Console:** A dynamic assessment interface featuring multiple-choice questions. It validates answers in real-time, instantly rewarding XP and adjusting user levels upon successful completion.

- **Star-Rating Leaderboard:** A dual-purpose system where students rate their learning experience (1-5 stars). These ratings aggregate onto a public leaderboard, fostering healthy competition among instructors.
- **Daily Auto-Attendance Logger:** An automated tracking system that logs student presence based on their first login or interaction of the day, particularly focusing on scheduled check-ins (e.g., 9:00 AM).
- **Admin Content Editors:** High-level interfaces allowing administrators to CRUD (Create, Read, Update, Delete) courses and oversee global platform health.

# 2. Technical Requirement Document (TRD)

## 2.1 Architecture Diagram & Description

AuraAI utilizes a robust, decoupled Client-Server architecture ensuring high availability, secure data transactions, and rapid content delivery. The frontend operates as a Single Page Application (SPA), communicating with a RESTful backend connected to a managed PostgreSQL database.




## 2.2 API Router Design

The backend Node.js/Express service exposes a structured RESTful API. Core endpoints include:

| Module | Endpoint Base | Description & Methods |
| --- | --- | --- |
| **Auth** | `/api/auth` | POST `/login`, POST `/register`, POST `/logout`. Handles JWT issuance via Supabase. |
| **Users** | `/api/users` | GET `/profile`, PUT `/profile`. Fetches XP, streaks, and roles. |
| **Courses** | `/api/courses` | GET `/`, POST `/`, GET `/:id`, DELETE `/:id`. CRUD operations based on role. |
| **Progress** | `/api/progress` | POST `/enroll`, PUT `/update`. Tracks module completion and XP accumulation. |
| **Quizzes** | `/api/quizzes` | GET `/:courseId`, POST `/submit`. Retrieves quiz data and validates answers. |
| **Attendance** | `/api/attendance` | POST `/check-in`, GET `/daily-list`. Logs the 9:00 AM attendance events. |

## 2.3 Security & Route Protection

Security is enforced both on the client and server. The client-side Vanilla JS router implements navigation guards. Before rendering an admin or instructor view, the router verifies the JWT payload residing in `localStorage` or `sessionStorage`. If a *Student* attempts to access `/admin`, the router intercepts the request and redirects to `/dashboard`.

Server-side, every protected route utilizes Express middleware to validate the Supabase Auth token and confirm Role-Based Access Control (RBAC). Direct API hits bypassing the UI are strictly blocked without valid token headers.

## 2.4 Performance & Error Handling

- **Non-blocking Requests:** The frontend utilizes native `fetch` API with async/await, ensuring the UI thread remains unblocked during network calls.
- **Offline Fallbacks:** Mock JSON files are stored locally during Phase 1. If network failure occurs, the SPA gracefully falls back to cached data, notifying the user of limited connectivity.

- **Mixed-Content Prevention:** All endpoints are strictly enforced over HTTPS. Assets and API requests are routed through secure protocols to pass modern browser mixed-content policies.

# 3. App Flow Document

The application logic maps discrete user journeys based on role authentication.

## 3.1 Student Flow

```
[Login via Auth Page]
  → [Onboarding / Interest Selection (First Login Only)]
    → [Student Dashboard: Displays XP, Streak, Recommended Courses]
      → [Browse Catalog & Enroll (Free or ₹ Checkout)]
        → [Lesson Module View: Video/Text Content]
          → [Interactive Quiz Console]
            → [Submit Score: Auto-Level Up & XP Gain]
              → [Post-Course: Rate Instructor 1-5 Stars on Leaderboard]
```

## 3.2 Instructor Flow

```
[Login via Auth Page]
  → [Role Check: Redirect to /instructor-portal]
    → [Instructor Dashboard: Daily Metrics & Reviews]
      → [Course Creator: Add Modules & Quiz Builder]
      → [Daily Attendance Tracker: View 9:00 AM Check-ins]
```

### 3.3 Admin Flow

```
[Login via Auth Page]
  → [Role Check: Redirect to /admin]
    → [Admin Overview: Global Login Stats]
      → [User Management: Create/Delete Student Accounts]
      → [Catalog Management: Inspect/Delete Flagged Courses]
```

# 4. UI/UX Design Document

## 4.1 Design Tokens & Typography

AuraAI utilizes a sleek, modern, and accessible color palette geared toward reducing eye strain during long learning sessions while highlighting critical actions.

- **Base Theme:** Dark Mode Native (Background: `#121212`, Surface Cards: `#1e1e1e`).
- **Accents:** Warning Yellow (`#facc15`) for alerts and notifications. Streak Orange (`#f97316`) for gamification elements like XP bars, daily streaks, and call-to-action buttons.
- **Typography:** System default sans-serif stack (Inter, Roboto, Segoe UI) optimized for legibility. Headings are bold and contrasting (White/Light Grey), while body text relies on softer greys (`#a1a1aa`).

## 4.2 Layout & Responsiveness

The layout is driven by Vanilla CSS leveraging CSS Grid and Flexbox (translated into block rendering principles where necessary for print). The UI features a fixed responsive sidebar for navigation, collapsing into a hamburger menu on mobile devices. Course lists are displayed in responsive grid cards. Checkout forms and profile edits are housed in centralized, focus-trapping modal popups. Micro-animations, such as a smooth hover-sweep effect on the 5-star rating component, enhance tactile feedback.

### 4.3 Navigation Controls

Navigation is highly contextual. The DOM dynamically hides or injects sidebar links based on the authenticated role. An Admin will see "Global Logs" and "Manage Courses", while a Student will only see "My Learning", "Leaderboard", and "Quizzes". This prevents UI clutter and enforces security by obscurity on the frontend (backed by rigid API security).

## 5. Database Schema Document

The Supabase PostgreSQL database follows a normalized relational structure. Note on data mapping: Database columns utilize `snake_case`, which the backend ORM/controllers map to `camelCase` JSON properties before sending to the client application.

## 5.1 Core Tables

**Table: `users`**

| Column Name | Data Type | Description |
|---|---|---|
| id | UUID (PK) | Unique user identifier linked to Supabase Auth. |
| username | VARCHAR(100) | Display name for the platform. |
| email | VARCHAR(255) | Unique email address. |
| role | ENUM | 'student', 'instructor', or 'admin'. |
| avatar | TEXT | URL to profile image. |
| bio | TEXT | Short biography (instructors/students). |
| xp | INT | Total experience points earned. Default 0. |
| level | INT | Calculated tier based on XP. |
| streak | INT | Consecutive days logged in. |
| rating | DECIMAL(3,2) | Average rating (instructors only). |
| rating_count | INT | Total number of ratings received. |

**Table: `courses`**

| Column Name | Data Type | Description |
|---|---|---|
| id | UUID (PK) | Unique course identifier. |
| title | VARCHAR(255) | Name of the course. |
| instructor_id | UUID (FK) | References users.id. |
| instructor_name | VARCHAR(100) | Denormalized for quick frontend rendering. |
| price | DECIMAL(10,2) | Cost in INR (₹). 0 for free. |
| xp_reward | INT | XP granted upon completion. |
| modules | JSONB | Array of lesson objects and media links. |
| quizzes | JSONB | Embedded array of Q&A objects for the console. |

**Table: `attendance`**

| Column Name | Data Type | Description |
|---|---|---|
| id | UUID (PK) | Unique log identifier. |
| user_id | UUID (FK) | References users.id. |
| username | VARCHAR(100) | Name of the student checking in. |
| date | TIMESTAMP | Exact timestamp of the check-in (e.g., 9:00 AM). |

# 6. Implementation Plan

## 6.1 Phased Execution Strategy

- **Phase 1: Local Development & Fallbacks.** Initialize the project directory. Construct the Vanilla HTML/CSS/JS frontend. Develop offline fallback mock files (JSON) for courses and users to allow UI testing without a live backend.
- **Phase 2: Database Setup (Supabase).** Provision the Supabase project. Execute SQL schema scripts to create `users`, `courses`, and `attendance` tables. Set up Row Level Security (RLS) policies based on the three roles.
- **Phase 3: Backend Deployment (Render).** Develop the Node.js/Express API. Connect the backend to the Supabase PostgreSQL connection string. Secure API keys via environment variables. Deploy the web service to Render.com and verify API health.
- **Phase 4: Frontend Deployment (Firebase).** Update the frontend JS router to point to the live Render API URLs. Configure Firebase CLI, build the static assets, and deploy to Firebase Hosting. Ensure redirect rules in `firebase.json` point all routes to `index.html` for SPA routing.
- **Phase 5: Verification & Walkthrough.** Execute the validation checksheet:
  1. Simulate an Admin logging in to create a test student.
  2. Simulate the test student logging in, enrolling in a course, and passing a quiz.
  3. Validate XP increment and ensure the 5-star rating updates the instructor's public average.
  4. Verify the 9:00 AM attendance logger accurately records the session.
  5. Ensure page redirect security prevents cross-role access.